

Real-Time Multisensor Data Acquisition and IoT Dashboard using FreeRTOS for Smart Environmental Monitoring

¹Dr. Arun Kumar Katkoori
ECE Department
CVR College of Engineering
Hyderabad, India
dr.arunkatkoori@gmail.com

²Dr. D. Bhanu Prakash
ECE Department
CVR College of Engineering
Hyderabad, India
pbhanududi@gmail.com

³Dr. Ravi Boda
ECE Department
Koneru Lakshmaiah Education
Foundation (KLEF), Hyderabad
raviu2015@klh.edu.in

Abstract— In this paper, an ESP32 microcontroller running FreeRTOS is used to design and implement a real-time environmental monitoring system. To gather vital environmental data, the system combines five sensors: gas (MQ3), temperature and humidity (DHT11), light (LDR), ultrasonic, and soil moisture. Priority-based scheduling and preemptive multitasking are made possible by FreeRTOS to effectively manage sensor tasks. Message queues are used to facilitate inter-process communication and guarantee synchronised data exchange between running tasks. Sensor data is remotely sent to a mobile interface through the Blynk IoT platform and shown locally on an OLED screen. The system is appropriate for smart agriculture, industrial safety, and environmental monitoring because it consistently performs with low latency, efficient task isolation, and real-time responsiveness. Future additions of sensors and machine learning modules for predictive analytics are also possible thanks to the modular design.

Keywords—FreeRTOS, Data acquisition, IPC, Multitasking, Sensors, Message Queue.

I. INTRODUCTION

More efficient data collection and analysis has been made possible by recent developments in embedded systems and the Internet of Things (IoT) for a range of real-world applications, such as smart cities, healthcare, and environmental monitoring [1]. These developments have encouraged the creation of systems that deliver precise and timely data, improving decision-making procedures and overall operational effectiveness. The open-source real-time operating system FreeRTOS is one such development that has proven to be a reliable way to manage multitasking in systems based on microcontrollers. For real-time embedded applications that demand dependable and deterministic behaviour, FreeRTOS is the perfect option because it provides effective task scheduling and inter-process communication (IPC) [2].

This paper's main objective is to develop and put into use a FreeRTOS-based multi-sensor data collection and monitoring system. To gather a variety of environmental data, this system combines a number of environmental sensors, such as a temperature sensor, a soil moisture sensor, an LDR sensor, an ultrasonic sensor, and a gas sensor [3]. These sensors enable the system to track a number of variables, including temperature, humidity, light intensity, distance, and gas concentration, which helps to improve environmental management.

Priority-based scheduling is a crucial component of this work since it guarantees that crucial duties, like keeping an eye on gas levels or temperature thresholds, are finished quickly and without interruption. The gathered sensor data is managed and presented using an IPC message queue, which facilitates seamless task-to-task communication [4]. In addition to being sent to a remote Blynk application via IoT for accessible, on-the-go monitoring, the processed data is shown on an OLED [5] screen for local, real-time monitoring. This dual-mode monitoring system is flexible and user-friendly, providing real-time environmental data from nearby and distant locations.

This paper aims to show how multi-sensor data can be effectively managed in a real-time embedded system and to emphasise the benefits of using FreeRTOS in combination with IoT technologies to improve data accessibility. The difficulties and possible solutions of combining several sensors and communication protocols into a responsive, integrated system are also examined in this study. The proposed system, which combines these technologies, provides an efficient way to monitor and control environmental conditions in real-time. This solution could be used in a number of industries, such as industrial automation, smart homes, and agriculture.

II. RELATED WORKS

Liu et al. [6] showed an embedded multi-sensor data fusion system for autonomous surface vehicle navigation using a real-time operating system, so stressing the need of RTOS in efficiently managing intricate sensor data. Designed to provide consistent real-time data processing in medical systems, Yukesh [7] developed a multi-sensor inertial data acquisition system for patient health monitoring. Kais et al. [8] developed a multi-sensor acquisition architecture emphasising the need of real-time systems in guaranteeing synchronised data collecting in order to benchmark sensor fusion techniques.

Kejun and Jianbo [9] presented an RTOS-based software architecture for multi-sensor fusion systems, demonstrating their value for modular and scalable data collecting solutions. Last but not least, Rockwood et al. [10] showed that an RTOS could manage several sensor activities at once in medical applications, so enabling multitasking in remote patient monitoring. Taken all together, these studies show how crucial RTOS is to enabling accurate, efficient, and real-time multi-sensor data processing and acquisition for a variety of uses.

III. PROPOSED METHODOLOGY

Five sensors total—temperature, soil moisture, LDR, ultrasonic, and gas—make up the proposed system. FreeRTOS is used for sensor data collecting and monitoring. Three main modules form the system design: a data processing and display module for real-time monitoring, a sensor module for data acquisition, and a task scheduling module for effective sensor task management. Linked to the microcontroller, every sensor loads the suitable drivers and calibration techniques to guarantee exact and consistent outputs.

Originally designed especially for embedded systems and microcontrollers, an open-source, real-time operating system known as FreeRTOS. It provides a portable and efficient multitasking platform that allows one to do several chores at once with exact timing and scheduling. FreeRTOS helps developers set task priorities and ensure that important processes are finished on schedule since it promotes cooperative and preemptive scheduling. Among its characteristics are message queues, semaphores, and mutexes—inter-process communication techniques that maximise data sharing and task synchronisation. FreeRTOS is very portable, scalable, and widely used in embedded systems, automation, and the Internet of Things because of its small footprint and dependability in real-time applications.

Concurrently being sent to the Blynk app via IoT, the processed data is shown on an OLED panel. Users using this dual-mode monitoring system get real-time feedback both locally on the OLED and remotely via the Blynk app. Less importance is paid to display and IoT communication tasks to avoid interfering with important sensor collecting processes. Extensive testing and validation is carried out to guarantee the accuracy, dependability, and performance under several scenarios of the system. Examined are metrics including task execution time, latency, and responsiveness to guarantee the system satisfies real-time operational needs. Emphasising FreeRTOS's advantages in managing multi-sensor systems with IoT capabilities, this ordered approach enables seamless sensor integration, fast job scheduling, and accurate data visualisation on both local and remote platforms.

Real-time systems depend critically on Inter-Process Communication (IPC), which enables data exchange and task or process coordination. In FreeRTOS, semaphores, message queues, and event groups help IPC to be attained. Among these, message queues are extensively applied for dependable and orderly data transfer between tasks in a structure. One job can send data while another retrieves it thanks to a message queue, so guaranteeing data integrity and synchronising. Working on a first-in- first-out (FIFO) basis, it supports prioritised communication to effectively manage important data. Thus, building responsive, modular, and well-coordinated embedded systems depends on IPC via message queues.

Under Free RTOS, each of the system's several connected sensors has a specific purpose while it constantly gathers data from all of them. A priority-based scheduling system ensures that important operations, including the gas sensor, are finished first, so enabling instantaneous alarms should hazardous gas detection arise. Every sensor activity gathers data and forwards it to an IPC message queue, where

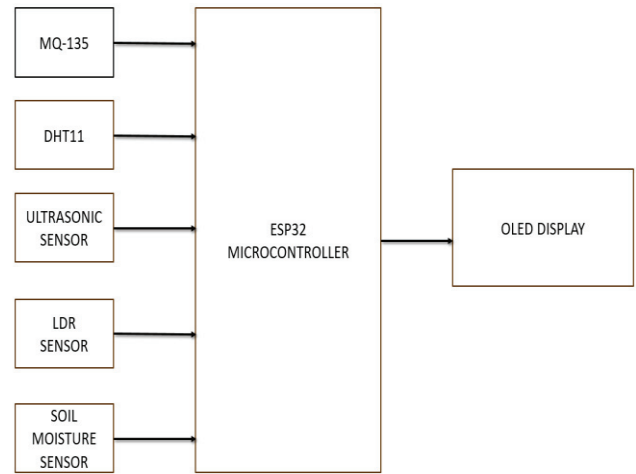


Fig. 1. Block diagram of FreeRTOS based multisensory data acquisition and display

it is momentarily stored to aid communication. Running a lower priority, the display and IoT communication activities retrieve data from the queue to refresh the OLED panel and notify the Blynk app over IoT in real-time. Users with this dual-monitoring system have local and remote access to environmental data.

The design of the system ensures that important sensor activities are not updates to displays or communication systems creating delays. Its modular architecture lets new sensors be seamlessly inserted and job priorities be changed to match application needs. For real-time applications needing local and remote monitoring, FreeRTOS enhances the general efficiency and responsiveness of the system, hence it is perfect.

IV. RESULTS AND DISCUSSION

As the schematic diagram shows, several sensors are connected to ESP32 in the following arrangements. Pin D22 on the ESP32 connects to the ECHO pin; pin D23 on the ultrasonic sensor from the TRIG pin. Pin D25 connects to the DHT11 temperature and humidity sensor's data pin, DATA. Pin D35 links the analog output (A0) of the gas sensor; pin D34 links the A0 of the soil moisture sensor. Pin D5 on the ESP32 is connected to LDR sensor digital output (D0). Every sensor runs on a single +5V power line and a GND connector.

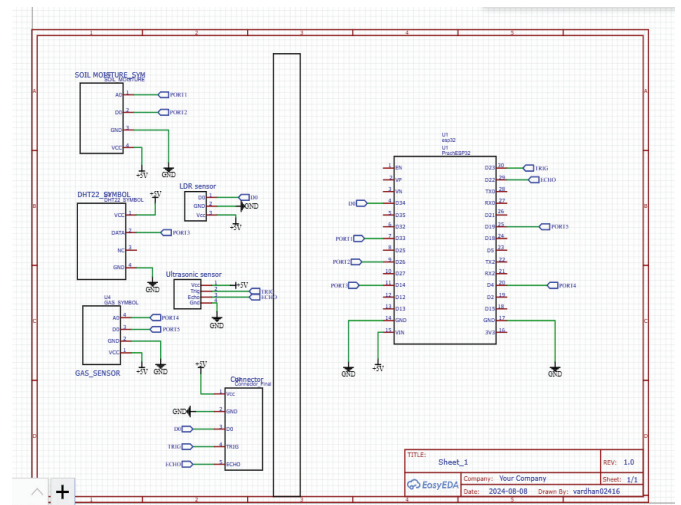


Fig.2. The schematic diagram of the model

The PCB design of above is shown in figure 3. By clearly marking each pin and component for simpler identification, the arrangement guarantees correct signal routing with little overlap. Connection routing is shown by red traces, and mounting holes are included for a safe installation. Because of the design's emphasis on modularity, real-time environmental monitoring applications may easily expand and link.

Figure 4 shows the Blynk dashboard showing the various sensors.

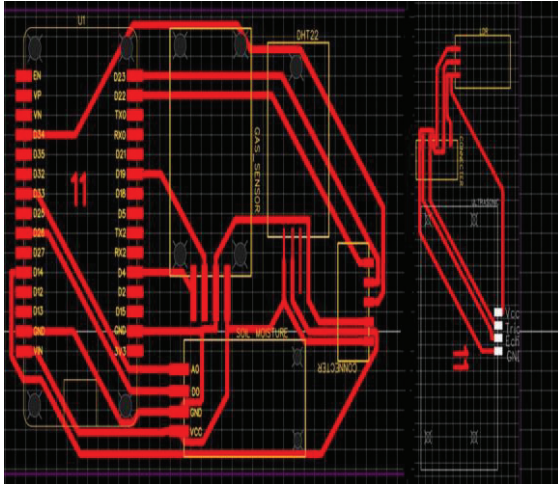


Fig. 3. PCB design

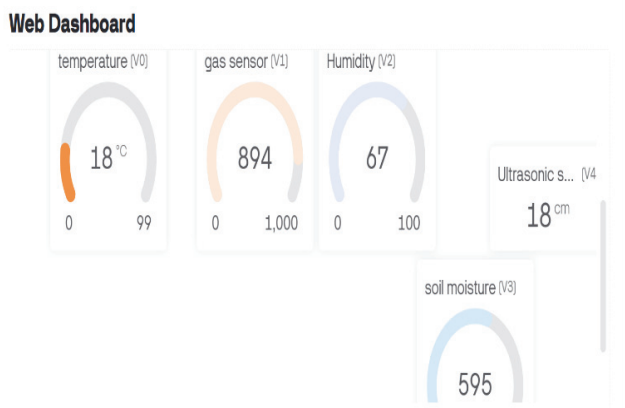


Fig.4. Blynk dash board showing the sensor values

Benchmarking was done in two configurations—with and without FreeRTOS—to assess the effect of FreeRTOS on performance. The findings unequivocally show that the RTOS-based system performs better in terms of CPU usage, task scheduling, and latency. This comparison is given in Table I.

For every sensor task, task execution times were recorded. High-priority jobs like gas detection were consistently finished in 3 ms in the FreeRTOS setup, but because of blocking delays and lack of preemption, execution times in the non-RTOS version finished in 8 ms. FreeRTOS's deterministic timing greatly aided the ultrasonic sensor task, which needed quick triggering and echo measurement. The FreeRTOS-based system had better task distribution, fewer idle or conflict states, and more efficient CPU utilisation, according to CPU profiling. On the other hand, under concurrent sensor activity, the polling-based method without FreeRTOS resulted in increased CPU load and sporadic data loss.

To dynamically modify task priorities in response to environmental conditions, the system also integrated a lightweight custom scheduler enhancement within FreeRTOS. For instance, the gas sensor task was elevated to top priority when dangerous gas levels were detected, starting a notification cycle right away and decreasing the frequency of lower-priority tasks like updating the soil moisture or OLED. Priority inheritance was used where needed to prevent priority inversion, a known problem in multitasking environments. This made sure that lower-priority tasks that were using shared resources, like message queues, wouldn't cause important tasks to be delayed. These enhancements highlight how crucial real-time operating systems are for embedded applications that need dependable, synchronised, and timely data collection from a variety of sensors.

TABLE I. PERFORMANCE COMPARISON – WITH AND WITHOUT FREERTOS

Metric	Without FreeRTOS	With FreeRTOS
Gas Sensor Execution Time	8 ms	3 ms
Ultrasonic Sensor Response	6 ms (variable)	2 ms (stable)
CPU Usage	80–90% (polling-heavy)	40–60% (efficient task management)
Message Handling Latency	>5 ms (manual sync)	2–3 ms (via message queue)
Task Scheduling	Sequential/blocking	Preemptive with priorities
Priority Inversion Handling	Not handled	Resolved using priority inheritance
Power Consumption	Higher (continuous polling)	Lower (condition-based triggering)
Expandability	Difficult	Modular and scalable

When using FreeRTOS as opposed to non-RTOS implementation, the figure 5 shows a notable decrease in task execution times for all sensor operations.

With FreeRTOS, the gas sensor task, which demands greater responsiveness, is finished in 2.8 ms as opposed to 8.5 ms without it, showing an improvement of more than 65%. Other sensors, such as the LDR and ultrasonic sensors, show comparable improvements. Preemptive multitasking in FreeRTOS, which enables time-sensitive tasks to execute with the least amount of delay, is responsible for these enhancements. This improves the system's responsiveness and qualifies it for real-time uses such as industrial safety systems and environmental monitoring.

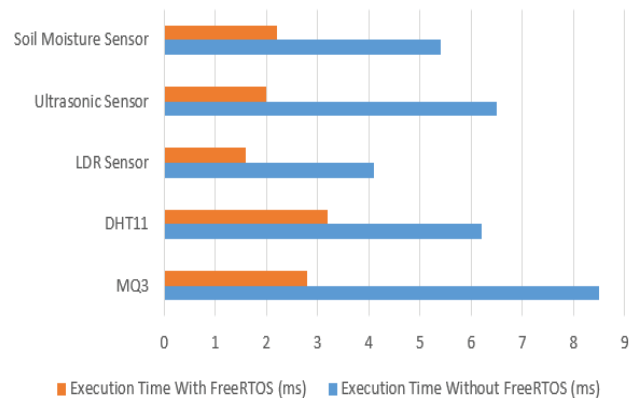


Fig.5. Task Execution Times (in ms) – With and Without FreeRTOS

V. CONCLUSION

Improved job scheduling, smooth inter-process communication, and effective multitasking are the outcomes of the paper's successful construction of a FreeRTOS-based multi-sensor system for real-time data collection and monitoring. The Blynk app's integration of IoT increased the system's functionality by enabling both local and remote real-time monitoring. The results validate the system's utility in smart agriculture, industrial safety, and environmental monitoring, among other applications. Simple extensions like adding more sensors, combining machine learning and predictive analytics, or enhancing IoT connectivity with advanced security features are made possible by its scalable and modular design. The potential for combining FreeRTOS and IoT to create robust, effective, and flexible embedded systems is illustrated in this paper.

REFERENCES

- [1] Padmaavathy, P.A., Bharathi, S.S., Arun Kumar Katkoori, Prasad, C.V.S., Ramachandran, G. (2022). Analysis of IoT Cloud Security Computerization Technology Based on Artificial Intelligence. In: Chen, J.I.Z., Tavares, J.M.R.S., Shi, F. (eds) Third International Conference on Image Processing and Capsule Networks. ICIPCN 2022. Lecture Notes in Networks and Systems, vol 514. Springer, Cham.
- [2] Melot, N. (2009). Study of an operating system: FreeRTOS. *CAPÍTULO XVIII*, 115.
- [3] Garud, R. R., & Mane, P. B. (2019). Smart farming using temperature sensor, moisture sensor, flow sensor and ultrasonic sensor leading to water conservation. *International Journal of Recent Technology and Engineering*, 8(3), 4426-4431.
- [4] Venkataraman, A., & Jagadeesha, K. K. (2015). Evaluation of inter-process communication mechanisms. *Architecture*, 86(64).
- [5] Geffroy, B., Le Roy, P., & Prat, C. (2006). Organic light-emitting diode (OLED) technology: materials, devices and display technologies. *Polymer international*, 55(6), 572-582.
- [6] Liu, Wenwen, et al. "The design of an embedded multi-sensor data fusion system for unmanned surface vehicle navigation based on real time operating system." 2018 OCEANS-MTS/IEEE Kobe Techno-Oceans (OTO). IEEE, 2018.
- [7] Yakesh, B. "Design of a multi-sensor inertial data acquisition system for patient health monitoring with real time operating system." 2016 International Conference on Advanced Communication Control and Computing Technologies (ICACCCT). IEEE, 2016.
- [8] Kais, Mikael, et al. "A multi-sensor acquisition architecture and real-time reference for sensor and fusion methods benchmarking." 2006 IEEE Intelligent Vehicles Symposium. IEEE, 2006.
- [9] Kejun, Zhang, and Su Jianbo. "RTOS-based software architecture for multisensor fusion system." 2004 5th Asian Control Conference (IEEE Cat. No. 04EX904). Vol. 2. IEEE, 2004.
- [10] Rockwood, Michael, et al. "Using a real-time operating system for multitasking in remote patient monitoring." 2014 IEEE International Symposium on Medical Measurements and Applications (MeMeA). IEEE, 2014.